

RandomForestClassifier#

[https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.h](https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html)

Description:

A random forest classifier. A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control overfitting. Trees in the forest use the best split strategy, i.e. equivalent to passing `splitter="best"` to the underlying `DecisionTreeClassifier`. The sub-sample size is controlled with the `max_samples` parameter if `bootstrap=True` (default), otherwise the whole dataset is used to build each tree. For a comparison between tree-based ensemble models see the example `Comparing Random Forests and Histogram Gradient Boosting models`. This estimator has native support for missing values (NaNs). During training, the tree grower learns at each split point whether samples with missing values should go to the left or right child, based on the potential gain. When predicting, samples with missing values are assigned to the left or right child consequently. If no missing values were encountered for a given feature during training, then samples with missing values are mapped to whichever child has the most samples.

Function Signature

```
class
sklearn.ensemble.RandomForestClassifier(n_estimators=100, *,
criterion='gini', max_depth=None, min_samples_split=2,
min_samples_leaf=1, min_weight_fraction_leaf=0.0,
max_features='sqrt', max_leaf_nodes=None,
min_impurity_decrease=0.0, bootstrap=True, oob_score=False,
n_jobs=None, random_state=None, verbose=0, warm_start=False,
class_weight=None, ccp_alpha=0.0, max_samples=None,
monotonic_cst=None)[source]#
```

Parameters:

`n_estimators`int, default=100

`criterion`{“gini”, “entropy”, “log_loss”}, default=“gini”

`max_depth`int, default=None

```
class
sklearn.ensemble.RandomForestClassifier(n_estimators=100, *,
criterion='gini', max_depth=None, min_samples_split=2,
min_samples_leaf=1, min_weight_fraction_leaf=0.0,
max_features='sqrt', max_leaf_nodes=None,
min_impurity_decrease=0.0, bootstrap=True, oob_score=False,
n_jobs=None, random_state=None, verbose=0, warm_start=False,
class_weight=None, ccp_alpha=0.0, max_samples=None,
monotonic_cst=None)[source]#
```

Parameters:

`n_estimators`int, default=100

`criterion`{“gini”, “entropy”, “log_loss”}, default=“gini”

`max_depth`int, default=None

`apply(X)`[source]#

Parameters:

X {array-like, sparse matrix} of shape (n_samples, n_features)

Returns:

X_{leaves} ndarray of shape (n_samples, n_estimators)

Parameters

ccp_alphanon

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. See Minimal Cost-Complexity Pruning for details. See Post pruning decision trees with cost complexity pruning for an example of such pruning. Added in version 0.22.

monotonic_cstarray

Indicates the monotonicity constraint to enforce on each feature. 1: monotonic increase 0: no constraint -1: monotonic decrease If `monotonic_cst` is `None`, no constraints are applied. Monotonicity constraints are not supported for: multiclass classifications (i.e. when `n_classes > 2`), multioutput classifications (i.e. when `n_outputs_ > 1`), classifications trained on data with missing values. The constraints hold over the probability of the positive class. Read more in the User Guide. Added in version 1.4.

`X{array}`

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

`X{array}`

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

`X{array}`

The training input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csc_matrix.

`yarray`

The target values (class labels in classification, real numbers in regression).

`sample_weightarray`

Sample weights. If None, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. In the case of classification, splits are also ignored if they would result in any single class carrying a negative weight in either child node.

paramsdict

Parameter names mapped to their values.

X{array}

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

X{array}

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

X{array}

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

Xarray

Test samples.

`yarray`

True labels for X.

`sample_weightarray`

Sample weights.

`**paramsdict`

Estimator parameters.

Code Examples

```
N_t / N * (impurity - N_t_R / N_t * right_impurity
           - N_t_L / N_t * left_impurity)
```

```
>>> from sklearn.ensemble import RandomForestClassifier
>>> from sklearn.datasets import make_classification
>>> X, y = make_classification(n_samples=1000, n_features=4,
...                           n_informative=2, n_redundant=0,
...                           random_state=0, shuffle=False)
>>> clf = RandomForestClassifier(max_depth=2, random_state=0)
>>> clf.fit(X, y)
RandomForestClassifier(...)
>>> print(clf.predict([[0, 0, 0, 0]]))
[1]
```

Notes & Warnings

NOTE See also `sklearn.tree.DecisionTreeClassifier` a decision tree classifier. `sklearn.ensemble.ExtraTreesClassifier` ensemble of extremely randomized tree classifiers. `sklearn.ensemble.HistGradientBoostingClassifier` Histogram-based Gradient Boosting Classification Tree, very fast for big datasets (`n_samples >= 10_000`).

Detailed Documentation

Documentation

A random forest classifier.

A random forest is a meta estimator that fits a number of decision tree classifiers on various sub-samples of the dataset and uses averaging to improve the predictive accuracy and control over-fitting. Trees in the forest use the best split strategy, i.e. equivalent to passing `splitter="best"` to the underlying `DecisionTreeClassifier`. The sub-sample size is controlled with the `max_samples` parameter if `bootstrap=True` (default), otherwise the whole dataset is used to build each tree.

For a comparison between tree-based ensemble models see the example [Comparing Random Forests and Histogram Gradient Boosting models](#).

This estimator has native support for missing values (NaNs). During training, the tree grower learns at each split point whether samples with missing values should go to the left or right child, based on the potential gain. When predicting, samples with missing values are assigned to the left or right child consequently. If no missing values were encountered for a given feature during training, then samples with missing values are mapped to whichever child has the most samples.

Read more in the [User Guide](#).

The number of trees in the forest.

Changed in version 0.22: The default value of `n_estimators` changed from 10 to 100 in 0.22.

The function to measure the quality of a split. Supported criteria are “gini” for the Gini impurity and “log_loss” and “entropy” both for the Shannon information gain, see [Mathematical formulation](#). Note: This parameter is tree-specific.

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than `min_samples_split` samples.

The minimum number of samples required to split an internal node:

If int, then consider `min_samples_split` as the minimum number.

If float, then `min_samples_split` is a fraction and $\text{ceil}(\text{min_samples_split} * n_samples)$ are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

If int, then consider `min_samples_leaf` as the minimum number.

If float, then `min_samples_leaf` is a fraction and $\text{ceil}(\text{min_samples_leaf} * n_samples)$ are the minimum number of samples for each node.

Changed in version 0.18: Added float values for fractions.

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

The number of features to consider when looking for the best split:

If int, then consider `max_features` features at each split.

If float, then `max_features` is a fraction and $\text{max}(1, \text{int}(\text{max_features} * n_features_in_))$

features are considered at each split.

If "sqrt", then $\text{max_features} = \sqrt{n_features}$.

If "log2", then $\text{max_features} = \log_2(n_features)$.

If None, then $\text{max_features} = n_features$.

Changed in version 1.1: The default of max_features changed from "auto" to "sqrt".

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than max_features features.

Grow trees with max_leaf_nodes in best-first fashion. Best nodes are defined as relative reduction in impurity. If None then unlimited number of leaf nodes.

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following:

where N is the total number of samples, N_t is the number of samples at the current node, N_{t_L} is the number of samples in the left child, and N_{t_R} is the number of samples in the right child.

N , N_t , N_{t_R} and N_{t_L} all refer to the weighted sum, if sample_weight is passed.

Added in version 0.19.

Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

Whether to use out-of-bag samples to estimate the generalization score. By default, `accuracy_score` is used. Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric. Only available if `bootstrap=True`.

For an illustration of out-of-bag (OOB) error estimation, see the example `OOB Errors for Random Forests`.

The number of jobs to run in parallel. `fit`, `predict`, `decision_path` and `apply` are all parallelized over the trees. None means 1 unless in a `joblib.parallel_backend` context. -1 means using all processors. See Glossary for more details.

Controls both the randomness of the bootstrapping of the samples used when building trees (if `bootstrap=True`) and the sampling of the features to consider when looking for the best split at each node (if `max_features < n_features`). See Glossary for details.

Controls the verbosity when fitting and predicting.

When set to `True`, reuse the solution of the previous call to `fit` and add more estimators to the ensemble, otherwise, just fit a whole new forest. See Glossary and `Fitting additional trees` for details.

Weights associated with classes in the form `{class_label: weight}`. If not given, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of `y`.

Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be `[[{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}]` instead of `[[{1:1}, {2:5}, {3:1}, {4:1}]`.

The “balanced” mode uses the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes *`

`np.bincount(y))`

The “balanced_subsample” mode is the same as “balanced” except that weights are computed based on the bootstrap sample for every tree grown.

For multi-output, the weights of each column of `y` will be multiplied.

Note that these weights will be multiplied with `sample_weight` (passed through the `fit` method) if `sample_weight` is specified.

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. See Minimal Cost-Complexity Pruning for details. See Post pruning decision trees with cost complexity pruning for an example of such pruning.

Added in version 0.22.

If `bootstrap` is `True`, the number of samples to draw from `X` to train each base estimator.

If `None` (default), then draw `X.shape[0]` samples.

If `int`, then draw `max_samples` samples.

If `float`, then draw `max(round(n_samples * max_samples), 1)` samples. Thus, `max_samples` should be in the interval `(0.0, 1.0]`.

Added in version 0.22.

1: monotonic increase

-1: monotonic decrease

If `monotonic_cst` is `None`, no constraints are applied.

multiclass classifications (i.e. when `n_classes > 2`),

multioutput classifications (i.e. when `n_outputs_ > 1`),

classifications trained on data with missing values.

The constraints hold over the probability of the positive class.

Read more in the User Guide.

Added in version 1.4.

The child estimator template used to create the collection of fitted sub-estimators.

Added in version 1.2: `base_estimator_` was renamed to `estimator_`.

The collection of fitted sub-estimators.

The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).

The number of classes (single output problem), or a list containing the number of classes for each output (multi-output problem).

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

Added in version 1.0.

The number of outputs when fit is performed.

The impurity-based feature importances.

Score of the training dataset obtained using an out-of-bag estimate. This attribute exists only when `oob_score` is `True`.

Decision function computed with out-of-bag estimate on the training set. If `n_estimators` is small it might be possible that a data point was never left out during the bootstrap. In this case, `oob_decision_function_` might contain `NaN`. This attribute exists only when `oob_score` is `True`.

The subset of drawn samples for each base estimator.

A decision tree classifier.

Ensemble of extremely randomized tree classifiers.

A Histogram-based Gradient Boosting Classification Tree, very fast for big datasets (`n_samples >= 10_000`).

The default values for the parameters controlling the size of the trees (e.g. `max_depth`, `min_samples_leaf`, etc.) lead to fully grown and unpruned trees which can potentially be very large on some data sets. To reduce memory consumption, the complexity and size of the trees should be controlled by setting those parameter values.

The features are always randomly permuted at each split. Therefore, the best found split may vary, even with the same training data, `max_features=n_features` and `bootstrap=False`, if the improvement of the criterion is identical for several splits enumerated during the search of the best split. To obtain a deterministic behaviour during fitting, `random_state` has to be fixed.

Breiman, "Random Forests", Machine Learning, 45(1), 5-32, 2001.

Apply trees in the forest to X, return leaf indices.

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

For each datapoint x in X and for each tree in the forest, return the index of the leaf x ends up in.

Return the decision path in the forest.

Added in version 0.18.

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

Return a node indicator matrix where non zero elements indicates that the samples goes through the nodes. The matrix is of CSR format.

The columns from indicator[n_nodes_ptr[i]:n_nodes_ptr[i+1]] gives the indicator value for the i-th estimator.

Build a forest of trees from the training set (X, y).

The training input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csc_matrix.

The target values (class labels in classification, real numbers in regression).

Sample weights. If None, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. In the case of classification, splits are also ignored if they would result in any single class carrying a negative weight in either child node.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

The predicted class of an input sample is a vote by the trees in the forest, weighted by their probability estimates. That is, the predicted class is the one with highest mean probability estimate across the trees.

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

The predicted classes.

Predict class log-probabilities for X.

The predicted class log-probabilities of an input sample is computed as the log of the mean predicted class probabilities of the trees in the forest.

The input samples. Internally, its dtype will be converted to dtype=np.float32. If a sparse matrix is provided, it will be converted into a sparse csr_matrix.

The class probabiliti
